

CS3307 Group Project Stage #1

Raspberry Pi Smart Dashboard Project Proposal

Group Number: 19

**Group Members: [Amaan Hafeez, Gary Han, Maotong Wei, Mike
Tran, Nael Pervez Lakhani]**

Table of Contents

1. Description.....	3
2. Features	3
2.1 Required Features (Minimum Viable Product)	3
2.2 Optional Features.....	4
2.3 Wish-list Features	4
3. Risks and Mitigation Strategies	5
3.1 Limited Processing Power of Raspberry Pi.....	5
3.2 Hardware Compatibility Issues.....	5
3.3 API Reliability and Rate Limits.....	5
3.4 Time Constraints and Scope Management	5
4. Conclusion	5
5. References	6

1. Description

The **Raspberry Pi Smart Dashboard** is a multi-functional display system designed to provide real-time system monitoring, weather updates, indoor and outdoor environmental data, a calendar with scheduled events, and a clock. The dashboard will be implemented using a **Raspberry Pi (preferably Pi 4 or Pi 5)** as the central processing unit, collecting and displaying various information through an intuitive and interactive user interface.

The software will be developed using **C++ and the Qt framework** for the graphical interface. Additional APIs and libraries will be used to retrieve weather data, system statistics, and synchronize calendar services. The system will support **four different display screens**, allowing users to switch between views dynamically.

Hardware Requirements:

- **Raspberry Pi** (Pi 4 or Pi 5)
- **Touchscreen display** or HDMI monitor
- **Sensors** for indoor/outdoor environment monitoring (e.g., temperature and humidity)
- **External API** or locally connected sensor module for fetching weather and environmental data

We will explore optional integrations such as **AI-powered voice commands and Spotify integration**, depending on project feasibility and available resources.

2. Features

2.1 Required Features (Minimum Viable Product)

- **Dashboard UI:** A user-friendly interface displaying various data widgets.
- **Raspberry Pi System Stats:** Display CPU usage, memory usage, temperature, and network status.
- **Weather Information:** Fetch and display real-time weather conditions and forecasts using an external API (e.g., OpenWeatherMap).

- **Indoor & Outdoor Environmental Data:** Display real-time temperature and humidity readings using DHT11/DHT22 or similar sensors.
- **Calendar & Time Display:** Synchronize and display calendar events from Google Calendar or another calendar API.
- **Settings Menu:** Allow users to configure dashboard preferences, such as API keys, update intervals, and screen settings.
- **Screen Navigation:** Ability to switch between four different screens dynamically.

2.2 Optional Features

- **AI Integration:** Use a basic AI chatbot or voice assistant for interacting with the dashboard.
- **Spotify Integration:** Display currently playing songs and allow basic controls (play, pause, skip) using the Spotify API.
- **Customizable UI:** Provide different themes or layouts that users can choose from.
- **Touchscreen Support:** Make the UI fully interactive for touchscreen displays.
- **Mobile Companion App:** A lightweight mobile app to sync with the dashboard and control settings remotely.

2.3 Wish-list Features

- **AI-powered Task Recommendations:** Analyze calendar events and suggest optimized schedules.
 - **Machine Learning-based Weather Predictions:** Implement a local model to predict weather trends.
 - **Energy Monitoring:** Use additional sensors to track electricity usage of connected devices.
 - **Gesture Control:** Use a motion sensor to allow gesture-based interactions with the dashboard.
-

3. Risks and Mitigation Strategies

3.1 Limited Processing Power of Raspberry Pi

- **Challenge:** The Raspberry Pi may struggle to handle multiple tasks and real-time API requests efficiently.
- **Mitigation:** Optimize code, use lightweight libraries, implement multi-threading, and introduce caching mechanisms for data retrieval.

3.2 Hardware Compatibility Issues

- **Challenge:** Integrating different sensors and modules may present driver or compatibility issues.
- **Mitigation:** Choose well-documented and widely supported hardware components.

3.3 API Reliability and Rate Limits

- **Challenge:** External APIs (e.g., weather, Spotify) may have usage limits or become unavailable.
- **Mitigation:** Implement local data storage for offline mode, use multiple API providers as backups, and minimize unnecessary API requests.

3.4 Time Constraints and Scope Management

- **Challenge:** Completing all planned features within the project timeline.
- **Mitigation:** Prioritize required features first and work on optional features only if time allows.

4. Conclusion

The **Raspberry Pi Smart Dashboard** will be a useful and interactive system integrating real-time data sources to help users manage their daily activities. The **modular and scalable** design will allow additional features to be incorporated over time. Our approach prioritizes **efficiency, usability, and feasibility**, ensuring that the core features are fully implemented while exploring optional integrations based on time and resource availability.

We look forward to developing a **versatile and extensible dashboard** suitable for both personal and professional use cases.

5. References

- OpenWeatherMap API Documentation: <https://openweathermap.org/api>
- Qt Framework Documentation: <https://doc.qt.io/>
- Google Calendar API Documentation:
<https://developers.google.com/calendar>